



ELTE
FACULTY OF
INFORMATICS

PROGRAMMING

Lecture 2

Zsuzsa Pluhár
pluharzs@inf.elte.hu

Refreshing



The course

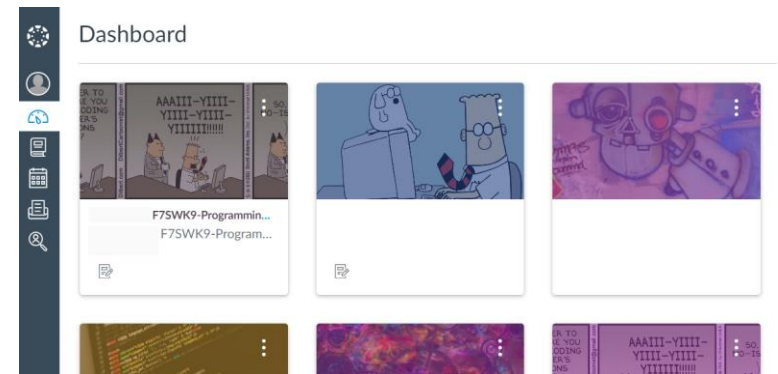
pluharzs@inf.elte.hu
room: 4.724

TOPIC

- basics of programming
- Theory of problem-solving – using patterns
- Programming language (C#) is only a tool to test

CANVAS.ELTE.HU

- tests, assignments
- requirements, RULES !!!
- deadlines



ELTE

FACULTY OF
INFORMATICS

PROGRAMMING – Zsuzsa Pluhár

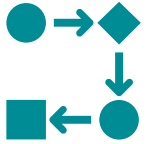
Specification



Components:

- **Input data** (identifier, set of values [unit of measure])
- Information about the input (**precondition**)
- **Output**, results (identifier, set of values)
- The statement – true for all input and output – used to get the result (**postcondition**)
- Definitions of the used concepts
- Requirements against the solution
- Restricting conditions

The algorithm



Elements of the algorithms:

- Sequence (step by step execution)
- Conditional branch (choice from 2 or more options based on a condition)
- Loop (repeat certain amount of times or until a certain condition is met)
- Subprogram (a complex activity, with a unique name – abstraction)

Loop



Loop

- repetitive execution
- In the everyday life:
 - recipe: *cook **until** boiling*
 - proverbs: *Don't cross the bridge **until** you come to it*
 - poems, lyrics, ... („Until I find him, I'm never gonna stop searching”, „We sleep until the sun goes down”)

I haven't found him yet?
search

Example - non-1 smallest divisor



Let's calculate the **non-1 smallest divisor** of an integer ($n > 1$)!

Specification:

In: $n \in \mathbb{N}$

Out: $d \in \mathbb{N}$

Pre: $n > 1$

Post: $1 < d \leq n$ and $d \mid n$ and $\forall i \in [2..d-1]: (i \nmid n)$

„for all i between 2 and $d-1$ is true that i does not divide n ”

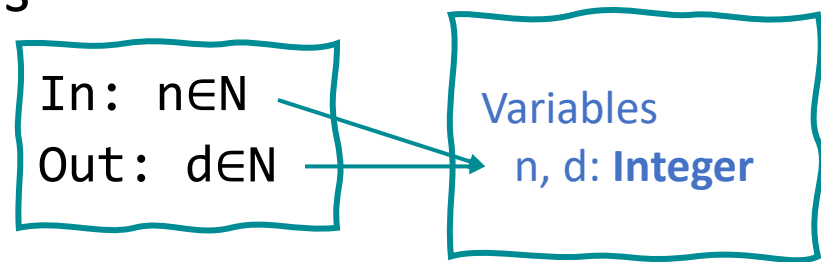
The $\nmid$ means not divisible:
a $\nmid$ b means a does not divide b



Example - non-1 smallest divisor

Representation of the solution:

Declaring program variables



„Rule” of representation when switching from specification to representation:

$\mathbb{N} \rightarrow \text{Integer (or variation)}$

Example - non-1 smallest divisor

An idea for the solution:

Let's try 2; if not good then try 3; if it's still not good then try 4, ..., worst case n will be a divisor.

in a conditional branch???

<i>true</i> \ <i>false</i>		$2 n$	
		<i>true</i>	<i>false</i>
$o:=2$	<i>true</i>	$3 n$	
	<i>false</i>	<i>true</i>	<i>false</i>
	$o:=3$	$4 n$	
	<i>true</i>	<i>true</i>	<i>false</i>
		$o:=4$	...

$i:=2$			
<i>true</i> \ <i>false</i>		$i n$	
		<i>true</i>	<i>false</i>
$i:=i+1$		$o:=i$	
<i>true</i> \ <i>false</i>		$i n$	
		<i>true</i>	<i>false</i>
$i:=i+1$		$o:=i$	
<i>true</i> \ <i>false</i>		$i n$	
		<i>true</i>	<i>false</i>
$i:=i+1$		$o:=i$	
...			

Example - non-1 smallest divisor

An idea for the solution:

Let's try 2; if not good then try 3; if it's still not good then try 4, ..., worst case n will be a divisor.

Algorithm expressing this:

Specification:

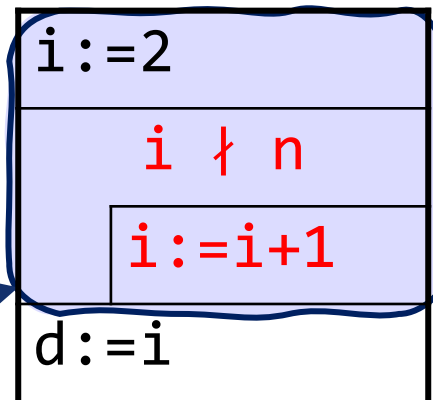
Input: $n \in \mathbb{N}$

Output: $d \in \mathbb{N}$

Precondition: $n > 1$

Postcondition: $1 < d \leq n$ and $d \mid n$ and

$\forall i (2 \leq i < d) : i \nmid n$



Variables
 $n, d: \text{Integer}$
 $i: \text{Integer}$

declaring the
local variable

The role of variable i : to go through numbers between 2 and n

Example – sd, gd

Task: Let's calculate both the **non-1 smallest** and the **non-n greatest** divisor of an integer ($n > 1$)!

Specification:

In: $n \in \mathbb{N}$

Out: $sd \in \mathbb{N}$, $gd \in \mathbb{N}$

Pre: $n > 1$

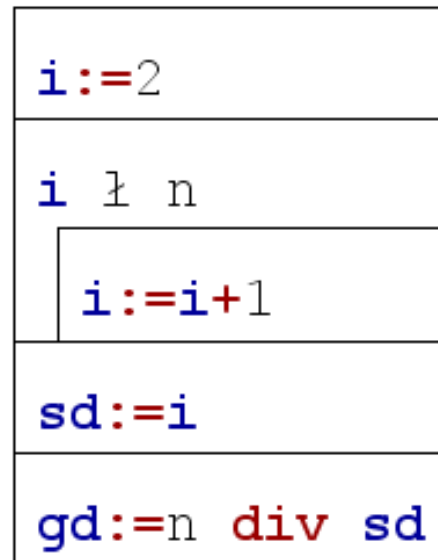
Post: $1 < sd \leq n$ and $1 \leq gd < n$ and
 $sd \mid n$ and $\forall i \in [2..sd-1]: (i \nmid n)$ and
 $gd \mid n$ and $\forall i \in [gd+1..n-1]: (i \nmid n)$

Example – sd, gd

Note:

By knowing sd and gd, the postcondition can be defined in another way: **$sd * gd = n$**

The **algorithm**:



Variables
n, sd, gd: **Integer**
i: Integer

Example – sd_2

Task: Let's calculate both the non-1 and the non-n smallest divisor of an integer ($n > 1$), if it exists!

Specification:

In: $n \in \mathbb{N}$

Out: $d \in \mathbb{N}$, $exists \in \mathbb{L}$

Pre: $n > 1$

Post: $exists = \exists i \in [2..n-1] : (i | n) \text{ and } exists \rightarrow (2 \leq d < n \text{ and } d | n \text{ and } \forall i \in [2..d-1] : (i \nmid n))$

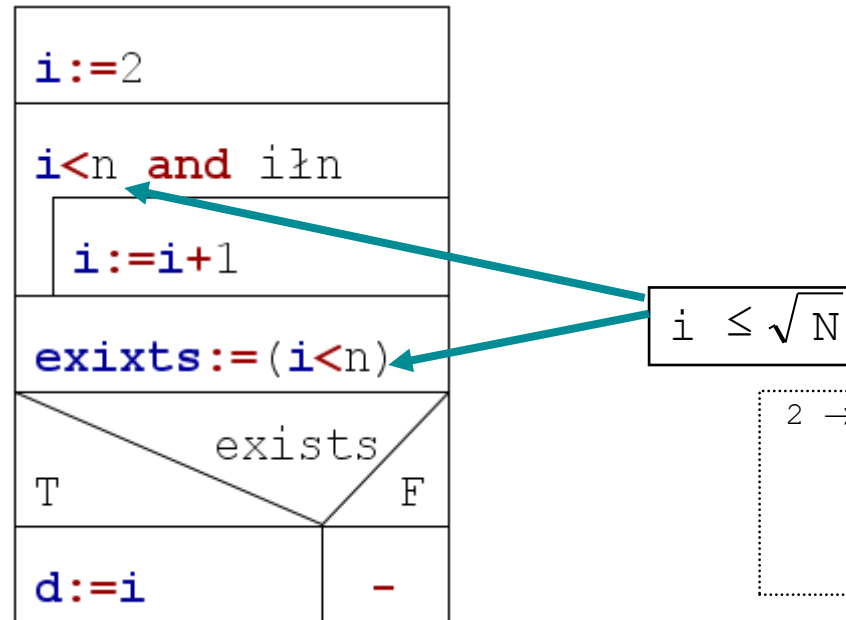
test cases:

$n=15 \rightarrow exists=true; d=3$

$n=17 \rightarrow exists=false; d=???$

Example – sd_2

Algorithm:



$2 \rightarrow i \leq (n \text{ div } i) \leftarrow n \text{ div } 2$
thus
 $i * i \leq n$
thus
 $i \leq \sqrt{n}$

Note:

Whenever i is a divisor of d , then $(n \text{ div } i)$ is also a divisor, so it's enough to check up to the square root of n

Example – sum of divisors

Task: Let's calculate the sum of all the divisors of an integer ($n > 1$)!

Specification:

In: $n \in \mathbb{N}$

Out: $s \in \mathbb{N}$

Pre: $n > 1$

Post: $s = \text{SUM}(i=1..n, i, i | n)$

$$s = \sum_{\substack{i=1 \\ i|n}}^n i$$

An example to understand conditional sum:

$N=15 \rightarrow$

$i=1 : (1 | 15) \rightarrow S=1$

$i=2 : (2 \nmid 15) \rightarrow S=1+0$

$i=3 : (3 | 15) \rightarrow S=1+3$

$i=4 : (4 \nmid 15) \rightarrow S=1+3+0$

...

$i=15 : (15 | 15) \rightarrow S=1+3+\dots+15$

Example – sum of divisors

Algorithm:

Specification:

In: $n \in \mathbb{N}$

Out: $s \in \mathbb{N}$

Pre: $n > 1$

Post: $s = \text{SUM}(i=1..n, i, i|n)$

$s := 0$

$i := 1..n$

$i | n$

T

F

$s := s + i$

-

we store the result in s variable

Question:

Could we stop looping at $\text{root}(n)$ again?

with $s := s + i + (n \text{ div } i)$ value assignment?



Example – sum of odd divisors

Task: Let's calculate the sum of all **odd** divisors of an integer ($n > 1$)!

Specification:

In: $n \in \mathbb{N}$

Out: $s \in \mathbb{N}$

Pre: $n > 1$

Post: $s = \text{SUM}(i=1..n, i, i | n \text{ and } \text{odd}(i))$

$$s = \sum_{\substack{i=1 \\ i|n \text{ and } \text{odd}(i)}}^n i$$

odd(i)=????



Example – sum of odd divisors

Task: Let's calculate the sum of all **odd** divisors of an integer ($n > 1$)!

Specification:

In: $n \in \mathbb{N}$

Out: $s \in \mathbb{N}$

Pre: $n > 1$

Post: $s = \text{SUM}(i=1..n, i, i | n \text{ and } \text{odd}(i))$

Definition:

$\text{odd}: \mathbb{N} \rightarrow \mathbb{L}$

$\text{odd}(x) := (x \bmod 2) = 1$

Example – sum of odd divisors

Algorithm₁:

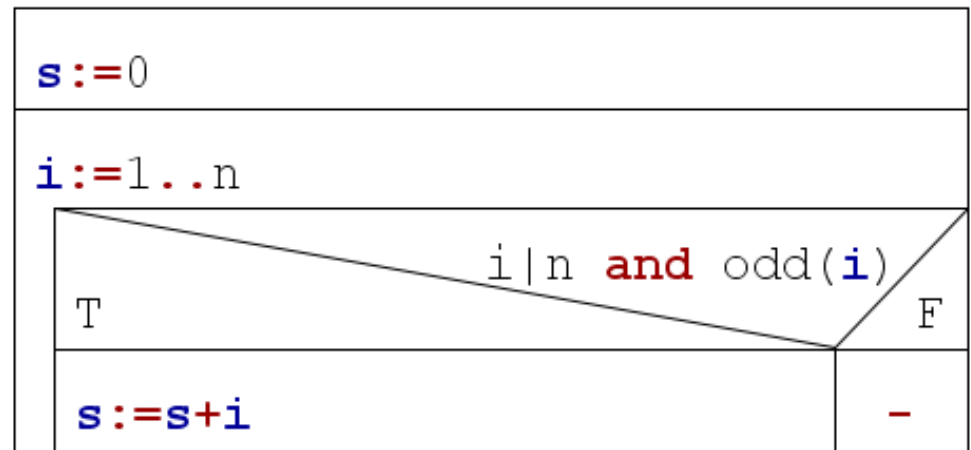
Specification:

In: $n \in \mathbb{N}$

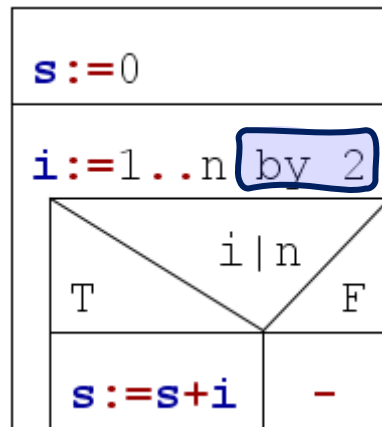
Out: $s \in \mathbb{N}$

Pre: $n > 1$

Post: $s = \text{SUM}(i=1..n, i, i|n \text{ and } \text{odd}(i))$



Algorithm₂:



Example – sum of prime divisors

Task: Let's calculate the sum of all **prime** divisors of an integer ($n > 1$)!

Specification:

In: $n \in \mathbb{N}$

Out: $s \in \mathbb{N}$

Pre: $n > 1$

Post: $s = \text{SUM}(i=1..n, i, i | n \text{ and } \text{prime}(i))$

$$s = \sum_{\substack{i=1 \\ i|n \text{ and } \text{prime}(i)}}^n i$$

prime(i)=????

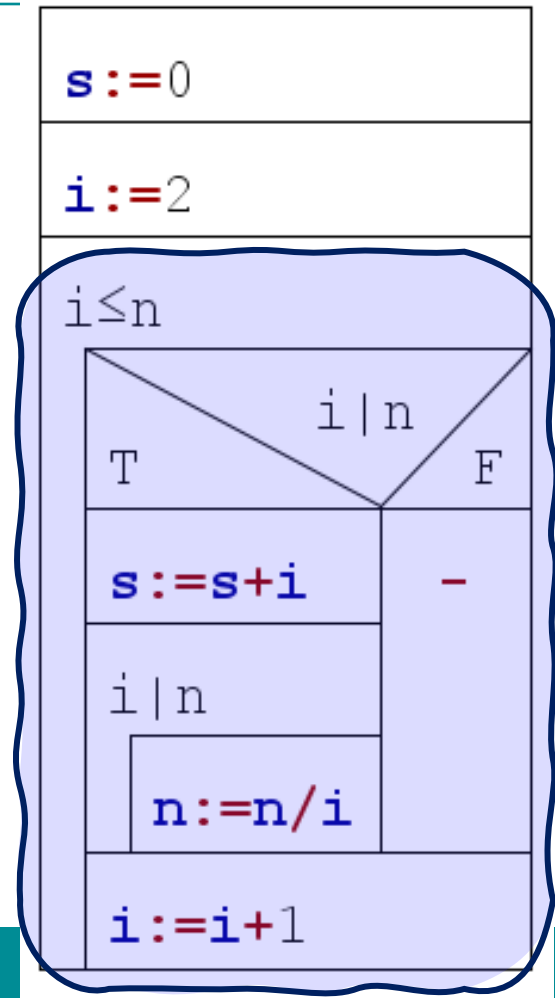


Example – sum of prime divisors

Algorithm:

- The *first* divisor is a prime for sure; let's divide n by it as many times as we can;
- the *next* divisor of the reduced n will be *prime again*.

Why don't we use a count loop as the outer loop?



Loops

Let's observe that:

- If there is $\exists$, $\forall$ or SUM (Σ sign) in postcondition, then the solution always contains a **loop**.
- If there is $\exists$, or $\forall$ sign in postcondition, then the solution is mostly a **conditional loop**.
- If there is SUM (Σ sign) in postcondition, then the solution is **mostly a for loop** (Π also).
- In the case of **conditional SUM (Σ)**, there will be a **conditional statement within the loop**.

Loop

conditional loop

condition
statements

```
while (condition){  
    statements;  
}
```

statements
condition

```
do{  
    statements;  
} while (condition);
```

counter loop

i=1..n
statements

```
for (int i=1;i<=n;++i){  
    statements  
}
```

i=1..n; by x
statements

```
for (int i=1;i<=n;i+=x){  
    statements  
}
```


$i := 2$			
$i < n$ and $i \neq n$			
<table border="1"> <tr> <td colspan="2">$i := i + 1$</td> </tr> </table>		$i := i + 1$	
$i := i + 1$			
$\text{exists} := (i < n)$			
T	F		
$d := i$	-		

code – loops, checking the precondition

```
độ độ
độ
Cộng độ Wsítê η
độ int TsýRắssê Cộng độ RắđđLíng độ η
độ độ η
ig độ
Cộng độ WsítêLíng Éssôss
xhíê độ
```

Sequence, array



Sequence

- **Sequence:** an ordered list of same typed data elements
- **Element:** reference to the i^{th} element of the sequence: $S[i]$
- **Index:** from `..to`, mostly: `1..SequenceLength`

Example:

- `month1 ∈ N[1..12]` – `month1` consists of 12 elements, indexed from 1 to 12, the elements are natural numbers $\cong (\text{month1}[1], \dots, \text{month1}[12])$,
- `floors ∈ S[-1..4]` – `floors` consists of 6 elements, indexed from -1 to 4, the elements are character strings $\cong (\text{floors}[-1], \text{floors}[0], \dots, \text{floors}[4]) = \text{basement}, \text{ground floor}, \dots)$



Array

- **Array:** finite length sequence, (the smallest and the greatest index, or the count of elements are known).
- **Index:** $1 \dots n$, sometimes $0 \dots n-1$, where n stands for the count of elements. Sometimes it is represented as $a \dots b$ ($a \leq b$).
- **Operations** on elements of an array:
 - referencing an element: $x[i]$
 - modification of an element (the element is selected by a given index): $x[i] := 3$

Sequence $\rightarrow$ Array

Example:

- In specification:

$n \in \mathbb{N}, \quad x, y, z \in R[1..n]$

$z[1] = x[1] + y[1]$

- example for declaration
- example for referencing

- In algorithm:

$x, y, z : \text{Array}[1..n : \text{Real}]$

$z[1] := x[1] + y[1]$

- example for declaration
- example for referencing

Using constant array

Task: Let's create a computer program, which writes an integer between 1 and 99 with letters.

Specification:

In: $n \in \mathbb{N}$,

$\text{ones} \in S[0..20] = ("", "one", \dots, "nineteen"),$

$\text{tens} \in S[0..9] = ("", "twenty", \dots, "ninety")$

Out: $s \in S$

Pre: $1 \leq n \leq 99$

Post: $n < 20 \rightarrow s = \text{ones}[n+1]$ and

$n \geq 20 \rightarrow \text{tens}[(n \text{ div } 10)] + \text{ones}[(n \text{ mod } 10) + 1]$

It's logical to put it here. From the point of the algorithm, the constant arrays are input.

Using constant array

Declaring program parameters

Variable

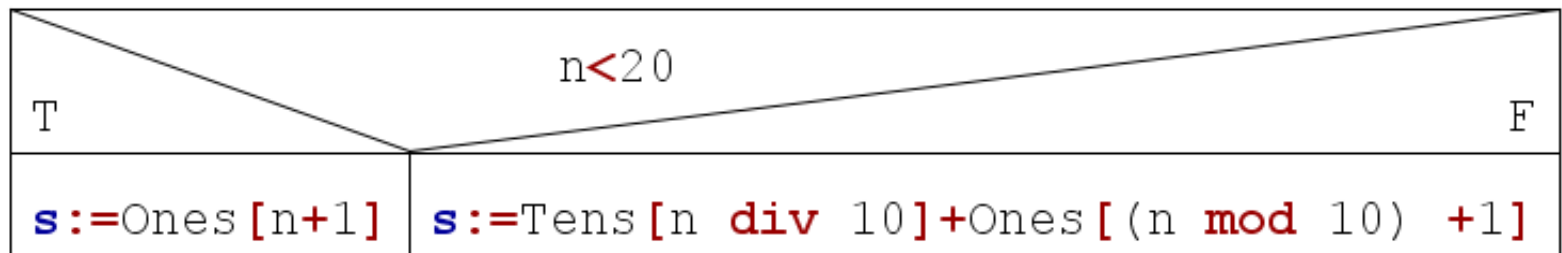
n:Integer

constant Ones:Array[1..20:String]=("", "one", ..., "nineteen")

constant Tens:Array[1..9:String]=("", "twenty", ..., "ninety")

s:String

Algorithm



> Code it!



ELTE

FACULTY OF
INFORMATICS

PROGRAMMING – Zsuzsa Pluhár

Example₂ – constant array

Task: Let's create a computer program that writes the serial number of a month based on the name of the month.

Specification:

In: $h \in S$,

$\text{monname} \in S[1..12] = (\text{"January"}, \dots, \text{"December"})$

Out: $s \in \mathbb{N}$

Pre: $h \in \text{monname}$

$\exists i \in [1..12] : \text{monname}[i] = h$

Post: $1 \leq s \leq 12$ and $\text{monname}[s] = h$

monname	
1	January
2	February
3	March
4	April
5	May
6	June
7	July
8	August
9	September
10	October
11	November
12	December

Example₂ – constant array

Declaring program parameters

Variable

h:String

s: Integer

constant MonName:**Array**[1..12:**String**]=(„January“,...,„December“)

Algorithm:

s := 1

MonName [**s**] ≠ h

s := s + 1

Question:

What would happen if the precondition was not met?
Runtime error? Infinite loop?



Example₃ – what do we store?

Task: We have a non leap year. What is the serial number of a given day (month, day)?

Specification:

In: $m \in \mathbb{N}$, $d \in \mathbb{N}$,

$\text{mon} \in S[1..12] = (31, 28, 31, \dots, 31)$

Out: $s \in \mathbb{N}$

Pre: $1 \leq m \leq 12$ and $1 \leq d \leq \text{mon}[m]$

Post: $s = \text{SUM}(i=1..m-1, \text{mon}[i]) + d$

mon	
1	31
2	28
3	31
4	30
5	31
6	30
7	31
8	31
9	30
10	31
11	30
12	31

Example₃ – what do we store?

Declaring program parameters

Variable

m, d, s : Integer

constant Mon:Array[1..12:Integer]=(31,28,31,...,31)

mon

1	31
2	28
3	31
4	30
5	31
6	30
7	31
8	31
9	30
10	31
11	30
12	31

Algorithm:

$s := d$

$i = 1 \dots (m-1)$

$s := s + \text{Mon}[i]$

Note:

In the case of a leap year, when $m \geq 3$, s should be increased by one. (The precondition should be modified, as well.)



Example₃ – what do we store?

Idea: Let's store how many days there are before each month

Specification₂:

In: $m \in \mathbb{N}$, $d \in \mathbb{N}$,

$\text{mon} \in S[1..12] = (0, 31, 59, \dots, 334)$

Out: $s \in \mathbb{N}$

Pre: $1 \leq m \leq 12$ and $1 \leq d \leq \dots$

Post: $s = \text{mon}[m] + d$

Question: Is this a better solution? How do we express the precondition?

mon	
1	0
2	31
3	59
4	90
5	120
6	151
7	181
8	212
9	243
10	273
11	304
12	334

array – algorithm → code

Some languages start indexing arrays at 0!

1ST IDEA: don't use the 0th item

algorithm		code	
		0	?
1	a	1	a
2	b	2	b
3	c	3	c

Declaration examples:

```
x:Array[1..n:Real]
```

in C#:

```
float[] x=new float[n+1];
```

Algorithm:

i=1..n
x[i] := i

in C#:

```
for (int i = 1; i <= n; i++) {  
    x[i]=i;  
}
```

The x[0] will be not used

array – algorithm → code

Some languages start indexing arrays at 0!

2ND IDEA: shift the index

algorithm		code	
1	a	0	a
2	b	1	b
3	c	2	c

Declaration examples:

`x:Array[1..n:Real]`

Algorithm:

$i = 1 \dots n$
$x[i] := i$

in C#:

```
float[] x=new float[n];
```

in C#:

```
for (int i = 1; i <= n; i++) {  
    x[i-1]=i;  
}  
//----- or -----  
for (int i = 0; i < n; i++) {  
    x[i]=i;  
}
```

array – algorithm → code

Some languages start indexing arrays at 0!

3RD IDEA: generalisation

Declaration examples:

`x:Array[b..e:Real]`

Algorithm:

$i = b \dots e$
$x[i] := i$

algorithm		code	
b	a	0	a
b+1	b	1	b
e	c	2	c

in C#:

```
float[] x=new float[e-b+1];
```

in C#:

```
for (int i = b; i <= e; i++) {  
    x[i-b]=i;  
}  
//----- or -----  
for (int i = b-b; i <= e-b; i++) {  
    x[i]=i;  
}
```


array – algorithm → code

Some languages start indexing arrays at 0!

3RD IDEA: example

Declaration examples:

`x:Array[-1..10:Real]`

Algorithm:

$i = -1 \dots 10$
$x[i] := i + 5$

algorithm		code	
-1	4	0	4
0	5	1	5
1	6	2	6

in C#:

```
float[] x=new float[12];
```

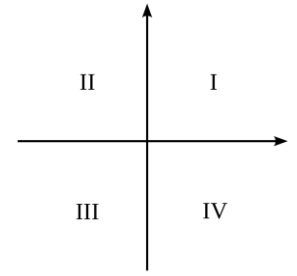
in C#:

```
for (int i = -1; i <= 10; i++) {  
    x[i+1]=i+5;  
}  
//----- or -----  
for (int i = 0; i <= 11; i++) {  
    x[i]=i+(-1)+5;  
}
```

Record, struct



Example – Point



Task: Let's determine which quadrant contains a given point!

Specification and algorithm

In: $x \in \mathbb{R}, y \in \mathbb{R}$

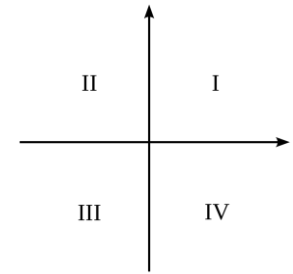
Out: $sn \in \mathbb{N}$

Pre: -

$x \geq 0$ and $y \geq 0$	$x < 0$ and $y \geq 0$	$x < 0$ and $y < 0$	$x \geq 0$ and $y < 0$
$sn := 1$	$sn := 2$	$sn := 3$	$sn := 4$

Post: $((x \geq 0 \text{ and } y \geq 0) \rightarrow sn=1)$ and
 $((x < 0 \text{ and } y \geq 0) \rightarrow sn=2)$ and
 $((x < 0 \text{ and } y < 0) \rightarrow sn=3)$ and
 $((x \geq 0 \text{ and } y < 0) \rightarrow sn=4)$

Example – Point



đề cấ sắ tậ iệ

độ cấ y. y

ệ tậ sậ

sắ đ iệ tậ iệ tậ

Cộ sộ lậ W sậ tậ y.

độ cấ T sậ Rắ sậ Cộ sộ lậ Rắ đ lậ ộ tậ y.

Cộ sộ lậ W sậ tậ y.

độ cấ T sậ Rắ sậ Cộ sộ lậ Rắ đ lậ ộ tậ y.

sộ lậ tậ iệ

iệ y. y. sậ ,

ê lậ iệ y. y. sậ ,

ê lậ iệ y. y. sậ ,

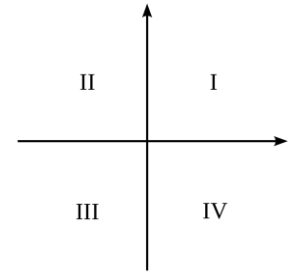
ê lậ iệ y. y. sậ -

x sậ iệ tậ ộ tậ tậ

Cộ sộ lậ W sậ tậ lậ Rắ đ sắ tậ sậ

$x \geq 0$ and $y \geq 0$	$x < 0$ and $y \geq 0$	$x < 0$ and $y < 0$	$x \geq 0$ and $y < 0$
sn:=1	sn:=2	sn:=3	sn:=4

Example – Point



Specification₂ and algorithm₂:

Postc: $(x \geq 0 \rightarrow (y \geq 0 \rightarrow sn = 1 \text{ and } y < 0 \rightarrow sn = 4))$

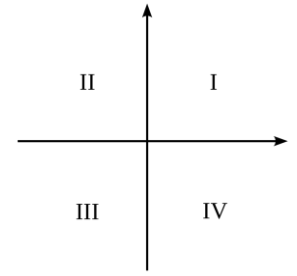
and

$(x < 0 \rightarrow (y \geq 0 \rightarrow sn = 2 \text{ and } y < 0 \rightarrow sn = 3))$

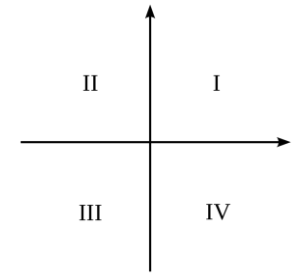
		$x \geq 0$	
		T	F
$y \geq 0$	T	$sn := 1$	$sn := 4$
	F	$sn := 2$	$sn := 3$

$\hat{y} \geq 0$
 $\hat{y} < 0$
 $\hat{y} \geq 0$
 $\hat{y} < 0$

Example – Point



- The solution
 - no semantic relationship between x and y
 - are not just two numbers, but together they denote a pair of coordinates
- Record:
 - the inclusion of data with different functions
 - creating a semantic unit
 - "function": what the data means (type)



Example – Point

Specification

- one data:
 - $x \in \mathbb{R}$
- mixed-data:
 - $p \in \text{Tpoint}, \text{Tpoint} = (\textcolor{red}{x}:\mathbb{R} \times \textcolor{blue}{y}:\mathbb{R})$
 - x : direct product
 - referencing: by the **name**
 - $p.\textcolor{red}{x}, p.\textcolor{blue}{y}$

definition of a new set (dataset)

the equality of sets

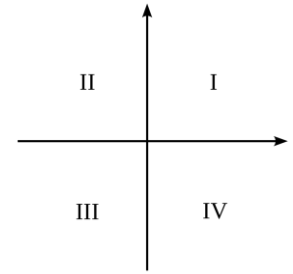
example for the direct product:

$a \in [1..2], b \in [4..5]$

a	b
1	4
1	5
2	4
2	5

lists all possible data pairs.

Example – Point



Task: Let's determine which quadrant contains a given point!

Specification_r and algorithm_r

In: $p \in \text{Tpoint}$,
 $\text{Tpoint} = (x:\mathbb{R} \times y:\mathbb{R})$

Out: $sn \in \mathbb{N}$

Pre: -

$p.x \geq 0$ and $p.y \geq 0$	$p.x < 0$ and $p.y \geq 0$	$p.x < 0$ and $p.y < 0$	$p.x \geq 0$ and $p.y < 0$
$sn := 1$	$sn := 2$	$sn := 3$	$sn := 4$

Post: $((p.x \geq 0 \text{ and } p.y \geq 0) \rightarrow sn=1) \text{ and}$
 $((p.x < 0 \text{ and } p.y \geq 0) \rightarrow sn=2) \text{ and}$
 $((p.x < 0 \text{ and } p.y < 0) \rightarrow sn=3) \text{ and}$
 $((p.x \geq 0 \text{ and } p.y < 0) \rightarrow sn=4)$

Record, struct

- define the struct by giving it a **name** and then adding the **member data types**
- *Structs* (also called *records*, or classes) group elements which don't need to all be the same type, and are **accessed by field (member) name**: $p=(p.x, p.y)$.
- Specification $\rightarrow$ algorithm

$p \in \text{Tpoint}, \text{Tpoint} = (x:R, x, y:R)$

Type Tpoint=**Record**(**x,y:Real**)

Variable $p:\text{Tpoint}$

Record, struct - code

type identifier: name of the new type

type identifier

data identifier

in C#:

```
struct TR{  
    public double x;  
    public double y;  
};
```

```
...  
TR a;  
...  
a.x=1;  
a.y=2;
```

field type

field identifier

Record, struct - code

şifşuçş TRỘîηş

řučl'îç đoučl'ê y ý

şţǎţîç wôîđ Nǎîŋ şţşîŋğ ăşğş

độc lập sắt iệt

độ ư ể ợ ỳ ỳ

Trở lại ở r

ing sh

sẽ đ³ing ìng

Cộng số lên 1 và viết tiếp

độốlê TsyRắssê Cộnsộlê RẻấđLîngê độtj r y.

Cộng số lên 1 và viết tiếp

độ ư c l e T s y R a s s e C o n s o l e R e a d L i n e ộ u t r y

Số lượng

îǧ ř ŷ

ř ý

၆၇

êl'sê îğ

ř y

•

ř

၇

êl'sê îğ

ř y.

○

ř

•

§

“

êl'sê îğ

ř y.

●

ř

 $\dot{y}$

၇

xsîţîŋğ ộựţ

Cộng số và Sinh nhị phân

Řuďďřňť

○

şη

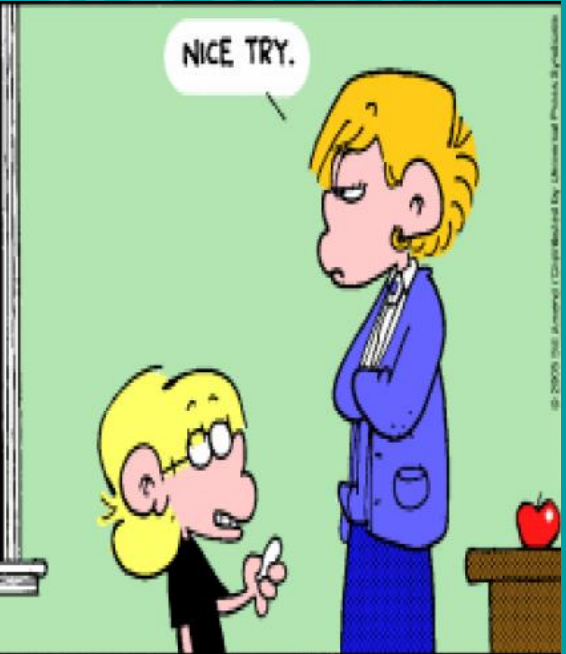
$p.x \geq 0$ and $p.y \geq 0$	$p.x < 0$ and $p.y \geq 0$	$p.x < 0$ and $p.y < 0$	$p.x \geq 0$ and $p.y < 0$
sn:=1	sn:=2	sn:=3	sn:=4



ELTE
FACULTY OF
INFORMATICS

```
#include <stdio.h>
int main(void)
{
    int count;
    for (count = 1; count <= 500; count++)
        printf("I will not throw paper airplanes in class.");
    return 0;
}
```

WEND 10-3



Thank you for your attention!